

46. Regarder à gauche ou à droite : le miroir

Un seul dessin, tourné vers la droite ; la console le retourne pour aller à gauche.

1. Ce qu'il faut comprendre

Un héros qui recule **de face** et avance de face à l'air de reculer. Il doit **regarder** où il va. On pourrait dessiner une seconde tuile, tournée vers la gauche ; il y a mieux : la console sait **retourner** un lutin, sans rien dessiner.

Ce qui est nouveau ici : le 5e argument de `sprite()`, `MIROIR_X`. `sprite(0, x, y, HEROS, MIROIR_X)` montre le dessin **retourné de gauche à droite**, comme dans un miroir. `MIROIR_Y` le retourne de haut en bas (un personnage qui tombe la tête en bas).

On range ce choix dans une **variable**, `regard` : `0` pour « tel quel » (vers la droite), `MIROIR_X` pour « retourné » (vers la gauche). `GAUCHE` met `regard = MIROIR_X`, `DROITE` le remet à `0`. On le passe à `sprite()` à chaque image.

Remarque ce qui se passe quand on **lâche** la flèche : rien ne remet `regard` à `0`. Le héros reste tourné **du côté où il allait**. C'est ce que font tous les jeux : une variable retient la dernière direction.

Le retournement ne coûte rien : c'est un bit dans les quatre octets qui décrivent chaque lutin à la console. Un seul dessin sert aux deux côtés, la moitié de la mémoire à dessiner.

À toi : avec `HAUT`, mets `regard = MIROIR_Y` et regarde ton héros la tête en bas.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **616 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      64 o ← la console, à l'allumage – la mise en route de la console
main()         100 o ← démarrage – ta fonction
VBlank         8 o ← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage  28 o ← image() – attend l'image suivante
AttendreVBlank 16 o ← démarrage – attend que l'écran soit libre
AttendreAcces  12 o ← EcrireTexte – attend de pouvoir écrire une case
EcrireTexte    10 o ← texte() – écrit un texte, lettre par lettre
RangerLutins   11 o ← démarrage – range les 40 lutins hors de l'écran
InstallerTransfert 15 o ← démarrage – installe la copie des lutins en mémoire rapide
CopierTuiles   85 o ← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage – vide les deux cartes de l'écran
LireManette    37 o ← bouton() – lit les huit boutons
Tuiles         192 o ← CopierTuiles – 11 lettres sur 43, et 1 tuile dessinée
DonneesTransfert 10 o ← InstallerTransfert – la petite routine qui copie les lutins
"GAUCHE DROITE" 14 o ← texte() – un texte gravé
```

Lettres gravées : ACDEGHIORTU

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `AdresseCase`, `EffacerCases`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `Notes`

Fonctions du langage appelées :

```
sprite() × 1    32 o sur place, dans main()
texte() × 1     11 o sur place, dans main() + EcrireTexte (10 o)
```

```
bouton() × 2    10 o sur place, dans main() + LireManette (37 o)
image() × 1     6 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

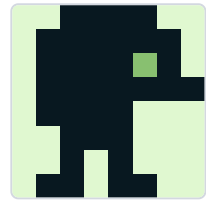
```
// CE PROGRAMME : un héros qui regarde du côté où il va : à droite avec DROITE,
// à gauche avec GAUCHE. Quand on lâche, il garde la dernière direction.
//
// Ce qui est nouveau : le 5e argument de sprite(), MIROIR_X.
// sprite(0, x, y, HEROS, MIROIR_X) montre le dessin retourné de gauche à droite.
// Un seul dessin sert donc pour les deux côtés.

// Le héros regarde à DROITE : son œil et son nez sont du côté droit.

#include <Tuile>    // un dessin de 8 × 8 pixels
#include <texte>    // écrit un texte à l'écran
#include <bouton>   // lit un bouton de la manette
#include <sprite>   // place un lutin de 8 × 8 au pixel près
```

2. UN DESSIN — HEROS

```
Tuile HEROS = {
    "..####..",
    "#####",
    "####-#.",
    "#####",
    "####...",
    "..###...",
    "..#.#...",
    "##.##..",
};
```



à l'écran

3. UNE VARIABLE — X

```
uint8_t x = 76;    // La place du héros, en pixels.
```

4. UNE VARIABLE — REGARD

```
uint8_t regard = 0;    // 0 : vers la droite (tel quel), MIROIR_X : vers la gauche
```

5. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {          // Le jeu commence ici.
    texte(1, 1, "GAUCHE DROITE");

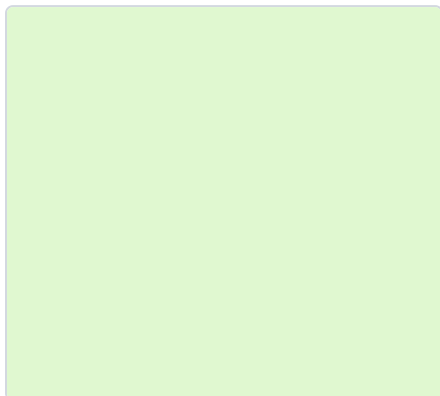
    while (true) {    // La boucle du jeu :
        image();

        if (bouton(DROITE) && x < 152) {
            x++;
            regard = 0;    // tel quel : il regarde à droite
        }
        if (bouton(GAUCHE) && x > 0) {
            x--;
            regard = MIROIR_X;    // retourné : il regarde à gauche
        }
        // Aucune flèche : rien ne change regard. Il garde la dernière direction.

        // Le 5e argument : 0 ou MIROIR_X, selon la dernière direction.
        sprite(0, x, 72, HEROS, regard);
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



DROITE — l'écran a changé



GAUCHE — l'écran a changé

5. Ce qu'on doit voir

Un héros tourné vers la droite ; GAUCHE le retourne, et il reste tourné vers la gauche quand on lâche.

6. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ au départ, le héros est tel quel
- ✓ GAUCHE le retourne
- ✓ il reste retourné quand on lâche
- ✓ DROITE le remet tel quel

